



img/Escudo_UNC.png

UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

CARRERA INGENIERÍA ELECTRÓNICA

**PROYECTO INTEGRADOR PARA LA OBTENCIÓN DEL
TÍTULO DE GRADO INGENIERO ELECTRÓNICO**

**“DISEÑO, DESARROLLO E IMPLEMENTACIÓN DE UN
SOFTWARE PARA LA ADQUISICIÓN Y ANÁLISIS DE ONDAS
DE IMPULSO TIPO RAYO (1,2/50 μ s).”**

Alumno

Tardón, Damián David

Director

Ing. Blasco, Marcos Javier

Co-Director

Ing Serra, Gabriel Horacio

Córdoba, República Argentina

Abril / 2026

Capítulo 1

Arquitectura del sistema

Este capítulo detalla el diseño estructural del software desarrollado para la adquisición y análisis de ondas de impulso tipo rayo ($1, 2/50 \mu\text{s}$). Se describe el patrón de diseño implementado para garantizar la escalabilidad y el desacople de los componentes, el flujo de datos desde el osciloscopio digital hasta la generación de reportes, y se presenta la justificación técnica del *stack* de desarrollo seleccionado en contraste con otras alternativas de la industria. Finalmente, se definen los lineamientos de gestión del entorno de desarrollo y el control de versiones aplicados durante el ciclo de vida del proyecto.

1.1 Patrones de arquitectura del software

Actualmente, para el desarrollo de software en general, se existen diversos patrones de diseño que ayudan a organizar el código, mejorar la mantenibilidad y facilitar la escalabilidad. Algunos de los patrones más comunes son: *Model-View-Controller* (MVC), *Model-View-Presenter* (MVP) y *Model-View-ViewModel* (MVVM), entre otros.

El patrón *Model-View-Controller* (MVC) es un patrón de diseño de software que se utiliza para separar la lógica de presentación, la lógica de negocio (procesar los datos, realizar cálculos y tomar decisiones) y la gestión de eventos en una aplicación. En este patrón, el **Modelo** representa los datos y la lógica de negocio, la **Vista** se encarga de la presentación visual y la interacción con el usuario, y el **Controlador** actúa como intermediario entre el Modelo y la Vista, gestionando las entradas del usuario y actualizando la Vista en consecuencia.

El patrón *Model-View-Presenter* (MVP) es similar al MVC, pero con una separación más clara entre la lógica de presentación y la lógica de negocio. En este patrón, el

Presentador se encarga de manejar la lógica de presentación y la interacción con el usuario, mientras que el **Modelo** sigue representando los datos y la lógica de negocio. La **Vista** se limita a mostrar los datos proporcionados por el Presentador y a enviar las acciones del usuario al Presentador.

El patrón *Model-View-ViewModel* (MVVM) es una evolución del MVP, el cual introduce el concepto de **ViewModel**, que actúa como un enlace entre el Modelo y la Vista. En este patrón, el ViewModel expone los datos y comandos que la Vista puede enlazar directamente, lo que facilita la separación de responsabilidades y mejora la testabilidad de la aplicación.

1.1.1 Arquitectura de software adoptada

El sistema se diseñó bajo el patrón arquitectónico *Model-View-Presenter* (MVP), con el objetivo de separar estrictamente la lógica de presentación (GUI), la lógica de negocio, y la gestión de eventos (presionar botones, ingresar texto o capturar una señal). Esta segregación es fundamental para cumplir con los requerimientos de validación algorítmica establecidos por la norma IEC 61083-2, permitiendo someter los módulos matemáticos a pruebas unitarias, utilizando los datos de calibración provistos en el software *Test Data Generator* o TDG, incluido en la norma, sin dependencia de la interfaz gráfica.

Los componentes de la arquitectura se distribuyen de la siguiente manera:

- **View (Vista):** Maneja exclusivamente la representación visual del software y la interacción del usuario, generada con la herramienta gráfica Qt Designer junto con la biblioteca PySide6, y la presentación de los gráficos mediante la librería pyqtgraph. La vista se mantiene completamente independiente de la lógica de adquisición y procesamiento, emitiendo señales (Qt Signals) que son capturadas por el presentador para desencadenar las acciones correspondientes.
- **Model (Modelo):** Contiene el manejo del hardware en GWInstekGDS1000AU, con su gestión de comandos SCPI vía PyVISA, en FileManager tiene la lógica de persistencia, encargado del almacenamiento jerárquico en formato HDF5 y exportación a CSV/XLSX, y en ImpulseAnalyzer se encuentra el motor de cálculo, que centraliza el procesamiento de señales, la aplicación de filtros digitales y el ajuste de curvas mediante el algoritmo de Levenberg-Marquardt, para extraer los parámetros de la onda (U_t , T_1 , T_2 , $OS\%$) exigidos por la norma IEC 60060-1.
- **Presenter (Presentador):** Implementado en la clase MainController. Actúa como orquestador conectando las señales emitidas por la Vista con los métodos (slots) correspondientes para interactuar con el Modelo, y actualizando la interfaz gráfica con los resultados procesados.

1.1.2 Manejo de Concurrency y Flujo de Datos

Dado que la comunicación con el osciloscopio (GW Instek GDS-1102A-U) requiere operaciones de entrada/salida (I/O) bloqueantes —específicamente durante la espera del disparo (*trigger*) del impulso—, la arquitectura incorpora un enfoque multihilo.

Se implementó la clase `WaitWaveformThread` heredada de `QThread`. Este hilo secundario sondea el estado del osciloscopio en segundo plano. Cuando se detecta el disparo, el hilo emite una señal (`wave_detected`) que es capturada por el hilo principal para proceder con la descarga del bloque de datos. Este diseño previene el congelamiento (*freezing*) de la GUI.

El flujo de datos continuo se define en cuatro etapas secuenciales:

1. **Adquisición:** El osciloscopio digitaliza la señal analógica. El bloque de datos binario es transferido a la PC vía USB (PyVISA), desempaquetado y convertido a un *array* de NumPy considerando los factores de atenuación del sistema de medición.
2. **Procesamiento:** Se almacena en el objeto `LightningImpulseAnalyzer` el *array* de tensión crudo. Se aplica la rutina indicada en el Anexo B de la norma IEC 60060-1, para obtener los parámetros característicos de la onda.
3. **Visualización:** Los *arrays* resultantes se inyectan en los objetos `PlotCurveItem` de `pyqtgraph` para su renderizado inmediato en pantalla.
4. **Persistencia:** La metadata del ensayo, las condiciones ambientales y los *arrays* de tensión (crudos y procesados) se empaquetan y almacenan en un archivo HDF5, garantizando la trazabilidad de los datos.

1.2 Elección del stack tecnológico

Para desarrollar el software de control del osciloscopio y procesamiento de los datos, se analizaron tres opciones que se consideran las más adecuadas para este trabajo: Python, C++ y LabVIEW. Estas opciones ofrecen herramientas y bibliotecas para la comunicación con instrumentos de medición.

Python es un lenguaje de programación de alto nivel, conocido por su simplicidad y legibilidad. Cuenta con una amplia gama de bibliotecas y frameworks que facilitan el desarrollo de aplicaciones científicas y de ingeniería. Además, es de código abierto, gratuito y de uso libre, incluso para fines comerciales, lo que permite acceder a una gran cantidad de recursos sin tener que pagar licencias.

C++ es un lenguaje compilado que destaca por su máximo rendimiento en tiempo de ejecución y control a bajo nivel de los recursos del sistema. Presenta una curva

de desarrollo pronunciada, la necesidad de gestionar la memoria manualmente, y la verbosidad de su sintaxis que incrementan drásticamente los tiempos de programación frente a lenguajes de mayor nivel de abstracción. Según el tipo de licencia que se adopte para QT, puede ser gratis o tener un costo asociado.

LabVIEW, es un entorno de desarrollo gráfico utilizado principalmente en aplicaciones de ingeniería y automatización, conocido por su facilidad para crear interfaces de usuario y su capacidad para interactuar con hardware de medición. Sin embargo, es un software propietario, con licencias costosas, que lo convierte en la opción menos accesible.

Las tres herramientas llevan mucho tiempo en el mercado y son ampliamente utilizadas en la industria, por lo que son opciones viables para llevar a cabo este proyecto. Sin embargo, dada la diferencia de costos y la agilidad de desarrollo requerida, Python se presenta como la opción más balanceada, accesible y eficiente para el contexto del laboratorio.

En la Tabla 1 se sintetiza el análisis comparativo que decantó en la adopción conjunto de herramientas: Python 3 para la lógica de negocio, PySide6 para la interfaz gráfica y PyVISA para la comunicación con el hardware.

Criterio de Evaluación	Python	C++	LabVIEW
Tipo de Lenguaje	Interpretado (Alto nivel).	Compilado (Alto nivel/Bajo nivel).	Gráfico (Flujo de datos).
Bibliotecas	Nativas + Externas. Múltiples aplicaciones.	Nativas + Externas. Enfocadas en rendimiento.	Nativas. Especializadas en hardware/instrumentos.
Desarrollo de GUI	QT Designer (<i>Drag & Drop</i>). Hay que codificar las funciones.	QT Designer (<i>Drag & Drop</i>). Hay que codificar las funciones.	Nativo (<i>Drag & Drop</i>). Funciones ya codificadas.
Costos	Gratis. <i>Open Source</i> (GPL/LGPL/BSD).	Depende de la licencia de Qt. <i>Open Source</i> o Comercial.	Licencia costosa.
Mantenimiento	Alto. Código basado en texto.	Alto. Código basado en texto.	Bajo. Archivos binarios (.vi), difícil revisión y actualización de código.

Tabla 1.1: Análisis comparativo de entornos para instrumentación y procesamiento de señales.

1.3 Bibliotecas utilizadas

Una biblioteca (o librería) en programación es un conjunto de código preescrito, funciones, clases y métodos reutilizables que permiten añadir funcionalidades específicas a las aplicaciones propias sin tener que escribir todo desde cero. Sirven para ahorrar tiempo, optimizar tareas comunes y estandarizar el desarrollo de software.

Una vez elegido Python como lenguaje de programación, a medida que se desarrolló el sistema, se incorporó las bibliotecas necesarias. Para ello se utilizó bibliotecas estándar, integradas en Python (Tabla 1.2), y de Terceros (Tabla 1.3).

Librería	Función general	Rol en el sistema
datetime	Manipular fechas y horas.	Generar de marcas de tiempo para nombres de archivos.
os	Usar la funcionalidades dependientes del sistema operativo.	Leer variables de entorno y manipular rutas del sistema.
pathlib	Representar rutas para cada sistema operativo.	Manejar rutas relativas y absolutas del sistema de archivos.
platform	Identificar el sistema operativo, la versión y la arquitectura.	Identificar el sistema operativo (Windows/macOS/Linux).
re	Analizar expresiones regulares.	Validar entradas del usuario en la GUI.
struct	Realizar conversiones entre valores de Python y estructuras de C.	Desempaquetar datos del osciloscopio para convertir a int/float.
subprocess	Ejecutar comandos del sistema operativo o aplicaciones externas.	Abrir el explorador de archivos.
sys	Gestionar la comunicación entre el script y el sistema operativo.	Ejecutar y cerrar procesos.
time	Gestionar retardos, duración de procesos y relojes del sistema.	Pausar ejecución en hilos o buffers de lectura.
typing	Especificar los tipos de datos en variables, funciones y métodos.	Tipado estático y declaración de estructuras de datos.

Tabla 1.2: Librerías integradas de Python.

Librería	Función general	Rol en el sistema
fpdf2	Generar documentos PDF.	Generar documentos de calibración en formato PDF.
h5py	Almacenar datos numéricos en formato binario HDF5, y manipularlos con NumPy.	Almacenar permanentemente los datos tipeados, digitalizados y/o calculados.
numpy	Manejar y procesar grandes volúmenes de datos.	Realizar el procesamiento digital de las señales.
openpyxl	Leer/escribir archivos Excel 2010 xlsx/xlsm/xltx/xltm.	Generar hoja de cálculo para exportar los resultados.
pandas	Analizar y manipular datos.	Estructurar datos para exportar resultados.
pyqtgraph	Visualizar datos y gráficos 2D/3D de alto rendimiento.	Graficar las señales adquiridas.
pyserial	Acceder al puerto serie.	Permitir la comunicación entre el osciloscopio y la PC.
PySide6	Crear aplicación de escritorio con interfaz gráfica de usuario (GUI).	Crear la GUI y manejar hilos/señales.
pytest	Automatizar la verificación de código, mediante pruebas unitarias.	Verificar el funcionamiento del código. Generar reporte de calibración del software.
pyvisa	Controlar instrumentos de laboratorio independientemente de la interfaz (GPIB, Serie, USB, Ethernet).	Establecer comunicación y control del osciloscopio vía protocolo VISA.
pyvisa-py	Backend para PyVISA.	Backend para PyVISA.
scipy	Incorporar funciones matemáticas complejas.	Realizar ajuste de curvas, y filtrado digital.

Tabla 1.3: Librerías externas de Python.

1.4 Gestión del entorno de desarrollo y control de versiones

El desarrollo del código fuente se centralizó en el entorno de desarrollo integrado Visual Studio Code (VS Code). Para asegurar la reproducibilidad del entorno de ejecución a través de diferentes estaciones de trabajo y evitar conflictos con los paquetes del sistema operativo, el proyecto se aisló utilizando entornos virtuales nativos (venv). Todas

las dependencias (ej. PySide6, numpy, h5py) quedaron registradas explícitamente en un archivo de requerimientos (`requirements.txt`) con sus versiones exactas. Esta homologación garantiza la utilización de las mismas librerías independientemente de la computadora de desarrollo, certificando así la estabilidad y el comportamiento determinista de los algoritmos matemáticos a lo largo del tiempo.

El control de versiones del código fuente se gestionó mediante Git. Se adoptó un flujo de trabajo basado en la creación de ramas. Esta estrategia permitió:

- Desarrollar características experimentales de forma aislada (ej. implementación de la GUI) en ramas secundarias.
- Integrar (*merge*) el código a la rama principal (`main`) únicamente tras la validación funcional de los módulos frente a las señales de referencia del TDG.